

TP 2 Analyse Numérique

La version interactive de ce TP se trouve ici : [ce lien](#)

Cette version interactive vous permettra de copier-coller le code python plus facilement !

Thèmes:

Décomposition LU, Systèmes linéaires, inverse de matrices, Equations différentielles.
Application

Consignes

Vous rendrez **A la fin de la séance** votre rapport en déposant votre fichier sur ce lien : [Dépôt TP2](#)

Si vous le souhaitez, vous pouvez poursuivre le travail chez vous et rendre un rapport amélioré avant le 1er juin 23h59. Votre note sera augmentée d'un maximum de 5 points

L'utilisation d'outils comme ChatGPT ou Copilot est autorisée **pour apprendre**, mais **toute production de texte/code générée doit être reformulée ou réécrite personnellement** dans le rapport final.

- Soupçons -> Division de la note par 2
- Certitude -> 0/20

Vous rendrez un rapport de TP avec

- les réponses aux questions,
- les parties de code que vous avez modifiées ou écrites en les expliquant,
- les graphiques que vous avez générés

Vous reprendrez la structure de ce sujet avec les titres et les sous-titres.

Vous pouvez travailler seul ou par binôme.

La décomposition LU

Nous avons établi en TD l'algorithme suivant pour obtenir la décomposition LU d'une matrice

```
import numpy as np

def DecomposeLU(A):
    n = len(A)
    L = np.eye(n) # Matrice identité
```

```

U = np.zeros((n,n)) # Matrice nulle

for i in range(n):
    # Calcul de la i-ème ligne de U
    for j in range(i, n):
        U[i,j] = A[i,j] - sum(L[i,k] * U[k,j] for k in range(i))
    # Calcul de la i-ème colonne de L
    for j in range(i+1, n):
        L[j,i] = (A[j,i] - sum(L[j,k] * U[k,i] for k in range(i))) /
U[i,i]

return L, U

```

Prenons la matrice

$$A = \begin{pmatrix} 1 & 1 & 2 \\ 2 & 1 & 1 \end{pmatrix}$$

qui peut s'écrire en python

```
A = np.array([[1, 1, 2], [1, 2, 1], [2, 1, 1]])
```

1. Vérifiez que la décomposition LU que nous avons calculée au TD 2 est correcte :

$$\begin{pmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \\ 2 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 2 & -1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 1 & 2 \\ 0 & 1 & -1 \\ 0 & 0 & -4 \end{pmatrix}$$

2. La décomposition LU d'une matrice A est :

$$L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 2 \end{pmatrix} \quad U = \begin{pmatrix} 1 & 1 & -1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Faites calculer la matrice A à python, puis recalculer L et U. Obtient-on bien le même résultat ?

Pour faire un produit matriciel, on utilise `A @ B` avec l'opérateur `@`, qui réalise le produit matriciel.

* Formez une matrice carrée aléatoire de taille $n = 3$ pour commencer. Mesurez le temps pour calculer la décomposition LU de cette matrice. Si vous êtes à l'aise, vous pourrez tracer le temps mis par la fonction en fonction de n .

Il existe des outils pour visualiser de grandes matrices en python, essayez par exemple

```
matplotlib.pyplot.imshow(A)
```

Résoudre des systèmes linéaires

En TD, nous avons utilisé la décomposition LU pour ramener tout système linéaire à des systèmes triangulaires.

Commençons donc par essayer de résoudre des *systèmes triangulaires* :

Si A est une matrice triangulaire supérieure, le système s'écrit

$$\begin{cases} a_{11} x_1 + a_{12} x_2 + \dots + a_{1n} x_n = b_1 \\ \phantom{a_{11} x_1} a_{22} x_2 + \phantom{a_{12} x_2} \dots + a_{2n} x_n = b_2 \\ \phantom{a_{11} x_1} \phantom{a_{22} x_2} \vdots \\ \phantom{a_{11} x_1} \phantom{a_{22} x_2} a_{n-1,n-1} x_{n-1} + a_{n-1,n} x_n = b_{n-1} \\ \phantom{a_{11} x_1} \phantom{a_{22} x_2} \phantom{a_{n-1,n-1} x_{n-1}} a_{nn} x_n = b_n \end{cases}$$

Pour résoudre ce genre de système, on commence par la dernière ligne puis on remonte les calculs :

$$\begin{cases} x_n = b_n / a_{n,n} \\ x_{n-1} = (b_{n-1} - a_{n-1,n} x_n) / a_{n-1,n-1} \\ x_{n-2} = (b_{n-2} - a_{n-2,n} x_n - a_{n-2;n-1} x_{n-1}) / a_{n-2;n-2} \\ \vdots \\ x_1 = (b_1 - a_{1,n} x_n - a_{1;n-1} x_{n-1} - \dots - a_{1,2} x_2) / a_{1,1} \end{cases}$$

Pour résoudre un système triangulaire supérieur, on peut donc utiliser :

```
def SolveTriSup(U, b):
    n = len(U)
    x = np.zeros(n)
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(U[i,j] * x[j] for j in range(i+1, n))) / U[i,i]
    return x
```

3. Ecrivez une fonction similaire `SolveTriInf(L, b)` pour résoudre un système triangulaire inférieur
4. Ecrivez une fonction python `Resoudre(A, b)` qui donne la solution du système linéaire $Ax = b$ (la fonction contient 3 lignes de code.)

Reprenons la matrice A dont la décomposition LU est :

$$L = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 2 \end{pmatrix} \quad U = \begin{pmatrix} 1 & 1 & -1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

5. Résoudre le système $Ax = b$ avec le vecteur $b = [3, 6, 8, 7]^T$.

6. Afficher la solution du système linéaire $AX = Y$ avec

$$A = \begin{pmatrix} 10 & 7 & 8 & 7 \\ 7 & 5 & 6 & 5 \\ 8 & 6 & 10 & 9 \\ 7 & 5 & 9 & 10 \end{pmatrix} \quad Y = \begin{pmatrix} 32 \\ 23 \\ 33 \\ 31 \end{pmatrix}$$

Faite de même si maintenant

$$Y = \begin{pmatrix} 32.1 \\ 22.9 \\ 33.1 \\ 30.9 \end{pmatrix}$$

Qu'en pensez-vous ?

Calcul de l'inverse

La décomposition LU permet aussi de calculer l'inverse d'une matrice.

Pour calculer l'inverse, on peut utiliser le programme suivant

```
def Inverse(A):
    L, U = DecomposeLU(A)
    n = len(A)
    N = np.zeros((n,n))
    M = np.zeros((n,n))
    for i in range(n):
        b = np.zeros(n)
        b[i] = 1 # b est la i-ième colonne de la matrice identité
        N[:,i] = SolveTriInf(L, b)
        M[:,i] = SolveTriSup(U, N[:,i]) # M est la matrice inverse de A
    return M``
```

7. Comparer la vitesse pour résoudre un système linéaire en calculant l'inverse ou en résolvant les deux systèmes triangulaires. Vous pourrez prendre de grandes matrices aléatoires.

8. Reprenez l'exemple précédent avec $AX=Y$. Calculez A^{-1} et expliquez le comportement de la question 6.

*Méthode de Jacobi :

Si vous n'êtes pas en avance, vous pouvez passer cette partie.

La méthode de Jacobi permet d'obtenir des solutions approchées d'un système avec une méthode itérative.

```
```python
def Jacobi(A, b, x0, tol=1e-6, max_iter=100):
 n = len(A)
 x = x0
 for k in range(max_iter):
 x_new = np.zeros(n)
 for i in range(n):
 x_new[i] = (b[i] - sum(A[i,j] * x[j] for j in range(n) if j !=
i)) / A[i,i]
 if np.linalg.norm(x_new - x) < tol:
 return x_new
```

9. Ecrivez une fonction qui calcule la distance entre deux vecteurs et tracer la distance entre les points intermédiaires de la méthode et la solution du système :

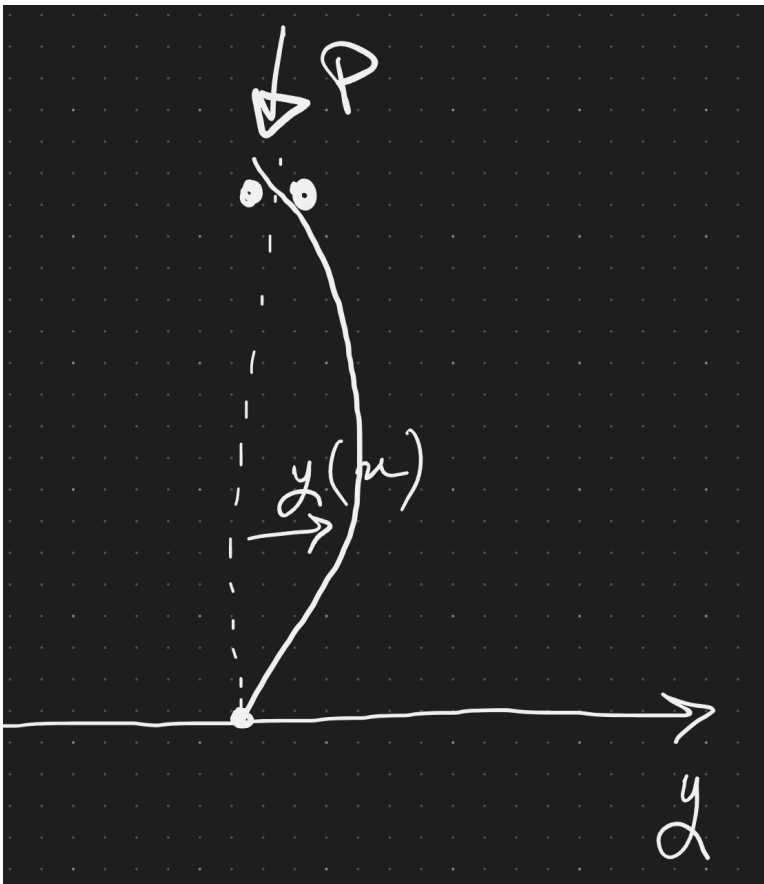
$$A = \begin{pmatrix} 4 & 1 & 1 \\ 2 & 5 & 1 \\ 1 & 1 & 6 \end{pmatrix} \text{ et } b = \begin{pmatrix} 9 \\ 15 \\ 21 \end{pmatrix}$$

10. Comparer la vitesse de la méthode Jacobi avec les méthodes de résolutions exactes des sections précédentes. Pour vous assurer de la convergence de la méthode de Jacobi, vous pourrez ajouter  $nI_n$  à une grande matrice aléatoire.

## Une application: phénomène de Flambage

On cherche à casser une règle en plastique en la pliant de 30 cm.

On note  $y(x)$  le déplacement transversal positif vers la droite.



L'équation différentielle est compliquée et fait intervenir le module d'Young et le moment d'inertie: les propriétés physiques d'une règle en plastique sont les suivantes :

Matériau pour une règle en plastique	Module d'Young (GPa)
<b>Polypropylène (PP)</b> – rigide bon marché, légèrement souple	1,4 – 1,6
<b>Polystyrène (PS)</b> – transparent, cassant	1,9 – 2,9
<b>ABS</b> – « incassable / flexible »	2,0 – 2,6

Pour le moment d'inertie, on modélise la section comme un rectangle :

- largeur (sens de lecture)  $b = 3\text{cm}$
- épaisseur  $h = 2,0\text{mm}$

a) Flexion dans le sens de la largeur, la règle est posée debout sur le sol, on la fait plier

$$I_z = \frac{b h^3}{12} = \frac{0,03 \times (0,002)^3}{12} \approx 2,0 \times 10^{-11} \text{ m}^4.$$

b) Flexion “de tranche”, la règle est couchée sur sa tranche.

$$I_y = \frac{h b^3}{12} = \frac{0,002 \times (0,03)^3}{12} \approx 4,5 \times 10^{-9} \text{ m}^4 (\simeq 200 \times \text{plus rigide}).$$

On exerce sur la règle une force verticale  $N$ .

11. A la main, de quel ordre de grandeur est la force que vous pouvez appliquer (en Newton) ?

Les rotations aux appuis sont libres :  $y(0) = y(L) = 0$ .

L'équation différentielle qui gouverne le déplacement est de la forme

$$EI \frac{d^2 y}{dx^2} + N y = q(x).$$

Pour trouver le moment où la règle va casser, on cherche une solution de l'équation en annulant le second membre. Nous nous intéressons donc à l'équation

$$EI \frac{d^2 y}{dx^2} + N y = 0.$$

En posant  $\alpha^2 = (N/EI)$ , l'équation s'écrit aussi :

$$\frac{d^2 y}{dx^2} = -\alpha y.$$

12. A l'aide des conditions aux limites, déterminer l'expression de  $y$  et la plus petite force de flambage qui fait casser la règle.

Nous allons faire tracer à l'ordinateur la solution de cette équation. On découpe la règle en  $n$  segments à l'aide de  $n + 1$  points

13. Donner la formule du pas spatial  $h$  et écrivez en python le tableau sommets qui contient les abscisses des points.

À l'aide de la différence centrée d'ordre 2,

$$\frac{d^2 y}{dx^2}(x_i) \approx \frac{y_{i-1} - 2y_i + y_{i+1}}{h^2},$$

on peut simplifier l'équation différentielle :

$$\frac{y_{i-1} - 2y_i + y_{i+1}}{h^2} + \alpha y_i = 0$$

14. Au brouillon, commencez par écrire les expressions obtenues pour  $n = 4$  puis généralisez. Écrire les équations pour chacun des noeuds, en remarquant que

$$y_0 = y_n = 0.$$

Ecrivez la collection d'équations sous la forme matricielle

$$A \mathbf{y} = \mathbf{b}.$$

15. Formez en python la matrice  $A$  puis utilisez les outils de la première section pour tracer

la solution  $y$ . Vous pourrez augmenter le nombre de segments pour obtenir une solution plus lisse.